

UM Bicycle & Scooter Rental Management System

Final Project Proposal (CISC3003)

Team Name: Team 01

Date: May 2026

Team Members

Pair	Student ID	Name	Role
09	dc325381	Hoi Kai Cheng	Leader Backend Architecture & Core API Logic
09	dc325107	Ho Weng Hong	Database Schema Design, Admin/Staff Module, Deployment Pipelines
07	dc325182	Che Chi Hin	Frontend Lead, UI/UX Design, Geofencing
07	dc325420	Chang I Wai	RWD CSS, Rate Limiting Logic
14	dc325095	Chung Hou Sam	Public Landing Pages, Documentation
14	dc325837	Lei Man Lam	i18n Framework, Backend Security

Table of Contents

UM Bicycle & Scooter Rental Management System.....	1
Team Members.....	1
Table of Contents.....	2
1. Project Background and Context.....	3
2. Project Objectives.....	3
3. Scope of Services.....	3
3.1 Student Services.....	3
3.2 Staff Services.....	3
3.3 Administrator Services.....	4
4. Functional and Data Requirements.....	4
4.1 Core Entities.....	4
4.2 Key Rules.....	4
5. System Architecture and Technology Stack.....	4
Frontend.....	4
Backend.....	4
Database.....	5
Tooling.....	5
6. Project Tasks and Milestones.....	5
Phase 1: Planning and Design.....	5
Phase 2: Core Development.....	5
Phase 3: Quality and Security.....	5
Phase 4: Deployment and Submission.....	5
7. Deliverables.....	5
8. Division of Work.....	6
9. Risk Assessment and Mitigation.....	6
10. Success Criteria.....	6
11. Evaluation Plan.....	7
12. Installation and Deployment Plan.....	7
Local Setup (XAMPP).....	7
Demo Reset.....	7
Container/Cloud Readiness.....	7
13. AI Usage Declaration.....	7
14. References (APA).....	8

1. Project Background and Context

Campus mobility is a recurring issue in the University of Macau (UM), especially for short-distance travel between hostels, academic buildings, and facilities. Existing transport options are either inefficient for short trips or lack centralized digital management.

This project proposes a centralized full-stack web platform for bicycle and scooter rental in UM. The system follows a LAMP/XAMPP architecture (PHP, MySQL, Apache), supports responsive web access across desktop and mobile devices, and includes role-based interfaces for Students, Staff, and Administrators.

The project is designed with real deployment in mind: local operation via XAMPP, containerized operation via Docker, and cloud-readiness for Google Cloud (Cloud Run + Cloud SQL).

2. Project Objectives

1. Improve Campus Mobility
Provide a reliable digital-first rental workflow for UM students.
 2. Enforce Strict Access Control
Implement RBAC boundaries among Student, Staff, and Admin roles.
 3. Deliver Full Localization
Support English, Simplified Chinese, and Traditional Chinese across public and authenticated pages.
 4. Ensure Operational Readiness
Separate demo and development data profiles for reproducible grading and testing.
-

3. Scope of Services

3.1 Student Services

5. Register/login and authenticated access
6. Vehicle search on interactive map
7. Start/End rental workflow
8. Wallet top-up and balance checking
9. Rental order history and status tracking
10. User profile management

3.2 Staff Services

11. Login to staff portal
12. Monitor fleet and station status
13. Manage student account statuses
14. Handle operational exceptions

3.3 Administrator Services

15. Manage staff/admin-related controls
 16. Resolve abnormal orders
 17. Perform force-close actions
 18. Manage key system entities (vehicles/stations/users)
-

4. Functional and Data Requirements

4.1 Core Entities

19. Users: campus ID, full name, role, password hash, email/phone, wallet balance
20. Vehicles: vehicle ID, type, brand, status, station assignment
21. Rental Stations: station ID, location, coordinates, capacity
22. Rental Orders: order ID, user ID, vehicle ID, start/end time, fee, status
23. Wallet Transactions: transaction ID, amount, method, timestamp, audit linkage
24. Rate-limit logs/security metadata for abuse prevention

4.2 Key Rules

25. Only authorized roles can access privileged routes.
 26. Sensitive files/config must not be web-accessible.
 27. Vehicle state transitions must remain consistent across student/staff/admin actions.
 28. Coordinates must stay within defined UM boundaries.
-

5. System Architecture and Technology Stack

Frontend

29. HTML5, CSS3, Vanilla JavaScript
30. Responsive Web Design (RWD)
31. Leaflet.js for map interaction

Backend

32. PHP 8+ for API/business logic/session handling

33. Role/session guards and validation

Database

34. MySQL relational schema

35. Seed/reset workflows (basic_seed.sql, seed_demo_data.sql, reset_demo_state.sql)

Tooling

36. XAMPP (local development)

37. Git/GitHub (version control)

38. Docker (containerization)

39. GCP SDK (deployment preparation)

6. Project Tasks and Milestones

Phase 1: Planning and Design

40. Requirements analysis

41. Use case diagram and ER design

42. Role and permission matrix

Phase 2: Core Development

43. Authentication and RBAC

44. Student dashboard and rental APIs

45. Wallet and order lifecycle

46. Staff/admin dashboards

Phase 3: Quality and Security

47. Rate limiting and API response hardening

48. i18n integration and translation coverage

49. State consistency fixes and regression checks

Phase 4: Deployment and Submission

50. Dockerization

51. Local reproducible setup scripts

52. Documentation, report, and PPT finalization

7. Deliverables

1. Web application source code (PHP/JS/CSS)
 2. Database scripts (schema, seeds, reset scripts)
 3. Technical documents (API_OVERVIEW.md, SECURITY_POLICY.md, TEST_CHECKLIST.md, DEPLOY_GCP.md)
 4. Deployment artifacts (Dockerfile, .dockerignore)
 5. Project report and presentation slides
 6. Demo screenshots and test-case evidence
-

8. Division of Work

53. Hoi Kai Cheng + Ho Weng Hong: architecture, APIs, DB schema, admin/staff backend, deployment pipeline
54. Che Chi Hin + Chang I Wai: frontend dashboard, UX/UI, map integration, RWD, rate limiting
55. Chung Hou Sam + Lei Man Lam: i18n, landing pages, security hardening, documentation system

All members participate in integration, testing, and submission review.

9. Risk Assessment and Mitigation

- API output corruption risk -> enforce JSON-safe error handling and try/catch wrappers
 - Data inconsistency risk -> standardized migration/seed/reset scripts
 - Map/geospatial errors -> coordinate normalization and strict boundary constraints
 - Role leakage risk -> centralized RBAC checks and route-level guards
 - Deployment delay risk -> local XAMPP and Docker fallback if cloud billing is pending
-

10. Success Criteria

- End-to-end student rental flow works with correct fee calculation
 - Unauthorized users cannot access admin operations
 - UI remains usable on desktop and mobile breakpoints
 - i18n toggles all target pages accurately
 - Demo reset scripts restore clean reproducible state
-

11. Evaluation Plan

- Functional test cases (happy path + edge cases + security checks)
 - Cross-role scenario testing
 - RWD checks on major viewport sizes
 - Data and status transition verification
 - Demo walkthrough with screenshot evidence
-

12. Installation and Deployment Plan

Local Setup (XAMPP)

7. Install XAMPP (Apache + MySQL + PHP 8+).
8. Place the project folder under htdocs.
9. Start Apache and MySQL.
10. Create a database via phpMyAdmin.
11. Import schema and seed scripts (basic_seed.sql or seed_demo_data.sql).
12. Run setup_database.bat if applicable.
13. Access via localhost and verify core flows.

Demo Reset

- Execute reset_demo_state.sql and re-import seed_demo_data.sql before demonstration.

Container/Cloud Readiness

- Build with Dockerfile.
 - Configure environment variables and DB connection.
 - Cloud deployment prepared for Cloud Run + Cloud SQL; public URL depends on billing activation.
-

13. AI Usage Declaration

AI tools (Google Gemini) may be used for language polishing, report structuring, debugging hints, and translation support. All final architecture, coding, testing, and submission decisions are made and validated by the student team.

14. References (APA)

Leaflet. (n.d.). *Leaflet*. <https://leafletjs.com/>

The PHP Group. (2026). *PHP: Hypertext Preprocessor*. <https://www.php.net/>

Apache Friends. (2026). *XAMPP*. <https://www.apachefriends.org/>

Oracle. (2026). *MySQL documentation*. <https://dev.mysql.com/doc/>

Docker, Inc. (2026). *Docker documentation*. <https://docs.docker.com/>

Google Cloud. (2026). *Cloud Run documentation*. <https://cloud.google.com/run/docs>

OWASP Foundation. (n.d.). *OWASP Top Ten*. <https://owasp.org/www-project-top-ten/>